The question remains of how the model could trade-off predictions from the induction head and from the feed-forward layer, which are added together due to residual connections. With fixed triggers Q, we may simply remove all $i \in Q$ from the summation in (8), so that the model exclusively relies on the attention head for all triggers (indeed, the output of W_O^2 is in the span of output embeddings, which are nearly orthogonal to the row space of W_F). When the triggers can vary across different sequences, choosing between the induction head and the feed-forward layer is more ambiguous as it depends on context, and W_F may try to learn more complex mappings that also use the outputs of W_O^2. In practice, we observe that the model often prefers the induction head, unless its output agrees with one of the top predictions from the global bigram, in which case it tends to prefer those.

Beyond the simplified architecture. While our simplified architecture already captures the relevant aspects for the bigram model, it lacks some of the components that appear in standard transformers, such as non-linear MLPs, trained embeddings, layer normalization, and joint learning of a factorization $W_K^\top W_Q$ (potentially with low rank matrices $W_K, W_Q \in \mathbb{R}^{d_h \times d}$ with $d_h < d$ as in multi-head attention), instead of a single matrix W_K. In practice, transformers also involve many more layers, as well as multiple heads at each self-attention layer. In Appendix D, we discuss how our memory viewpoint naturally extends to such architectural components, and we illustrate in Appendix E that they empirically lead to similar observations. Nonetheless, we focus on our simpler architecture in the main paper due to simplicity of exposition and better interpretability thanks to a clear identifiability of the role of each matrix, which is lost in models with more heads and layers.

5 Empirical Study

In this section, we present our empirical analysis of learning dynamics on the bigram data defined in Section 3, for the simplified architecture defined in Section 4.2. See Appendix E for additional results. Our code is available at `https://github.com/albietz/transformer-birth`.

Experimental setup. We train our models using mini-batch SGD with momentum, where each batch consists of 512 fresh sequences of length $T = 256$ sampled from our synthetic model. We use a fixed learning rate and weight decay. Hyperparameters are given in Appendix E. Unless otherwise noted, we use $d = 128$, random triggers with $\pi_q = \pi_u$ and uniform output tokens. The reported accuracies and losses are computed over each fresh batch before it is used for optimization, and are averaged over relevant tokens: "in-context accuracy/loss" numbers only consider predictions of output tokens on triggers starting at the second occurrence (the first is non-deterministic), while "global loss" refers to average loss on non-trigger tokens.

Memory recall probes. In addition to loss and accuracy, we consider metrics to check whether individual matrices have learned the desired associative memories: for a desired target memory $W_* = \sum_{(i,j)\in\mathcal{M}} v_j u_i^\top$, the corresponding recall metric is computed from the empirical estimate $\hat{W}$ as

$$R(\hat{W}, W_*) = \frac{1}{|\mathcal{M}|} \sum_{(i,j)\in\mathcal{M}} \mathbb{1}\{\arg\max_{j'} v_{j'}^\top \hat{W} u_i = j\}. \tag{9}$$

We use this for each matrix in (7) as target, and additionally test the previous token matrix W_K^1 on smaller time windows. For the final feed-forward layer, we measure the average KL divergence between the predicted softmax distribution using only W_F and the global bigram distribution π_b:

$$d_{KL}(W_F, \pi_b) := \frac{1}{N} \sum_{k=1}^{N} d_{KL}(\sigma(W_U W_F w_E(k)), \pi_b(\cdot|k)). \tag{10}$$

Emergence of the induction head via top-down learning. We begin our study by only training to minimize the loss on *trigger-output* token predictions after their first occurrence. This should be predictable with 100% accuracy using the two-layer induction head mechanism according to Section 4. We also remove the feed-forward layer, in order to focus on the learning of attention matrices W_K^1, W_K^2 and W_O^2 in isolation.

Figure 3 studies the effect of freezing different layers until iteration 300 on the training dynamics. By looking at memory recall probes, we see that training key-query matrices does not lead to any

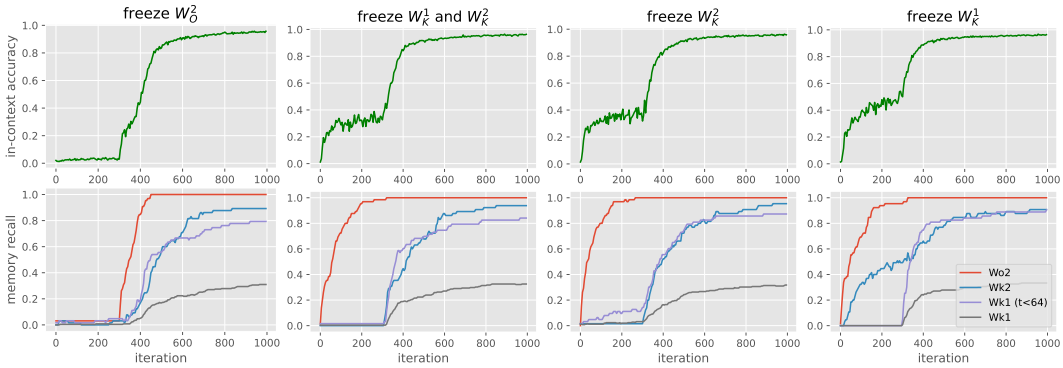

Figure 3: **Learning the induction head alone: in-context accuracy (top) and recall probes (bottom)** with some layers frozen until iteration 300. The output matrix W_O^2 can and must be learned before the key-query matrices, but does not suffice for good accuracy. It is easier to learn W_K^2 before W_K^1, and W_K^1 stores initial context positions ($t < 64$) much faster than late positions.

learning unless W_O^2 is learned first, and that W_O^2 can learn the correct associations even when trained by itself with key-value matrices at random initialization. Recall that the attention weights are essentially uniform when W_K are at random initialization, so that training W_O^2 alone resembles a bag-of-words models that aggregates representations throughout the sequence. While such a model has poor prediction accuracy, it is nevertheless sufficient to recover the correct associations in W_O^2 (a similar observation was made in [50] in a different setup).

Then, these associations enable learning key-query matrices that focus the attention on relevant tokens, by storing relevant key-query pairs in the form of associative memories, which eventually recovers the desired induction head behavior and leads to near-perfect accuracy. The two rightmost plots suggest that the second layer is learned before the first, in the sense that W_K^2 is easier to learn when W_K^1 is frozen compared to the reverse, yet learning them together seems beneficial, possibly due to helpful feedback loops [2]. We also observe that W_K^1 fits previous token associations for early positions much faster than later positions (purple vs gray line). This is likely due to the fact that it should be enough for the previous token head to attend to the first appearance of each trigger q_k, which is typically early in the sequence, so that most of the gradient will focus on early positions.

Overall, this provides a fine-grained understanding of the learning dynamics of induction heads. In Section 6, we analyze how a few gradient steps in a top-down fashion may suffice to recover appropriate associative memories in high dimension and with enough data. See Appendix E for additional experiments, including on the role of dimensionality.

Global vs in-context learning. Figure 4(left/right) shows that when training all layers jointly, the global bigram statistics tend to be learned more quickly than the induction head, as seen from the quick drop in loss and KL in early iterations. The W_O^2 probe also seems to improve quickly initially, but only leads to mild improvements to in-context predictions. The full learning of the in-context mechanism takes longer, likely due to slower dynamics of the key-query matrices. We also observe a tension between W_O^2 and W_F later in training, leading to slight degradations of our probe metrics. This may be due to the fact that the input to W_F now contains additional signal from the induction head which may be leveraged for better predictions, in particular for disambiguation in the case of random triggers, so that our guess of memories in Section 4.2 may no longer be accurate.

Role of the data distribution. We can see in Figure 4(left) that changes to the data distribution can have a significant effect on the speed of learning the in-context mechanism. We observe that the following may slow down in-context learning: (i) a smaller number of triggers K, (ii) using only rare fixed triggers, and (iii) using random triggers instead of fixed triggers. By inspecting the individual memory probes (see Figure 5 in Appendix E), we hypothesize that (i) and (ii) are due to slow learning of W_O^2, while (iii) is more related to slow learning of key-query matrices. This is reasonable since (i-ii) reduce the number of overall output tokens in the data, while (iii) increases the number of possible trigger tokens that should be stored in W_K^2, thus increasing the data requirements in order to learn the full associative memory. We also show in Figure 4(center) that changing the

Figure 4: **Global vs in-context learning and data-distributional effects.** (left) Loss on global (dashed) vs in-context (solid) tokens throughout training, for fixed or random trigger tokens q_k. The red curves fixes the trigger q_1 to the most frequent token, while the fixed triggers in blue curves are less common. (center) In-context accuracy with different training and test distributions π_o for output tokens. Uniform leads to better generalization than global bigrams π_b. (right) Probe metrics throughout training: W_O^2 and W_F eventually compete and deviate from our natural estimates.

output token distribution to bigram distributions at training time reduces the in-context accuracy when using out-of-distribution output tokens, while training on uniform outputs performs well on both distributions. This highlights that using a more diverse training distribution can lead to models with better generalization accuracy, with little additional training cost.

Additional experiments. In Appendix E, we provide additional experimental results for varying dimensionality, more complex architectures and training methods, as well as more fine-grained visualizations of the memory associations.

6 Theoretical Insights on Learning Dynamics

In this section, we provide theoretical insights on how gradients near initialization may allow the emergence of induction heads, and how this behavior is affected by data-distributional properties.

Finding signal in noisy inputs. In Lemma 1, we showed how gradient dynamics on a simple classification task with fixed embeddings of the inputs and outputs lead to associative memories. We now show that when inputs consist of superpositions of multiple embeddings, as is the case in the transformer residual streams, gradients may learn associative memories that filter out irrelevant components of these superpositions, focusing on useful signal instead.

Lemma 2 (Gradient associative memory with noisy inputs)**.** *Let p be a data distribution on $(x, y) \in \mathbb{R}^d \times [N]$, and consider the following classification problem, with fixed output embeddings W_U:*

$$L(W) = \mathbb{E}_{(x,y)\sim p}[\ell(y, W_U W x)].$$

The gradients take the following form: denoting $\mu_k := \mathbb{E}[x|y = k]$ and $\hat{\mu}_k := \mathbb{E}_x[\frac{\hat{p}_W(k|x)}{p(y=k)} x]$,

$$\nabla_W L(W) = \sum_{k=1}^{N} p(y = k) w_U(k)(\hat{\mu}_k - \mu_k)^\top.$$

The key takeaway from this lemma is that with enough data (here infinite data), the associative memory arising from gradients can learn to filter out noise from inputs, since it only depends on its expectations or conditional expectations. In particular, μ_k can isolate relevant parts of x that are predictive of a label k, and thus can lead to the right associations.

An illustrative example. To gain more intuition about this result, consider the following example: we would like to predict y from $x = w_E(y) + p_t$, where p_t is a positional embedding at a random position $t \in [T]$, which we would like to ignore. Further assume that y is uniformly distributed with $p(y = k) = 1/N$, and consider the matrix obtained after one population gradient step with step-size η starting from an initialization $W_0 = 0$ (so that $\hat{p}_{W_0}(k|x) = 1/N$):

$$W_1 = \frac{\eta}{N} \sum_{k=1}^{N} w_U(k)(\mu_k - \bar{\mu})^\top,$$